



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Python para Ciencia de Datos

Tema 6 Ficheros

Máster F.P. En IA & Big Data ANALYTICS
Curso 2026-2027

Resultados de aprendizaje

- Explicar el concepto de archivo (fichero)
- Identificar las diferencias entre archivos binarios y de texto
- Implementar programas que puedan leer el contenido de un archivo de texto
- Implementar programas que puedan crear, guardar y anexar información a archivos de texto
- Gestionar adecuadamente los errores que pueden producirse durante la gestión de archivos

Índice



- 1. Introducción**
- 2. Apertura y cierre de archivos**
- 3. Lectura de archivos de texto**
- 4. Manejo de errores**
- 5. Escritura de archivos de texto**

Índice



- 1. Introducción**
- 2. Apertura y cierre de archivos**
- 3. Lectura de archivos de texto**
- 4. Manejo de errores**
- 5. Escritura de archivos de texto**

Introducción

¿Por qué necesitamos ficheros?

- Los ficheros (o archivos) permiten a los programas **almacenar** datos de forma **permanente** en el disco (o en cualquier otro tipo de memoria no volátil)
- De esta forma, permiten que la información se **conserv**e, incluso después de que finalice el programa o se apague el ordenador
- Mediante ficheros también es posible **compartir datos** entre diferentes programas o entre diferentes ejecuciones del mismo programa

Introducción

En este tema nos centraremos en la gestión de **archivos de texto**

Archivos de texto vs binarios

- Los **archivos de texto**:
 - Contienen texto sin formato, que es legible por humanos
 - Consisten en una **secuencia de caracteres** organizados en **líneas**
 - Por lo general, se codifican mediante una cierta codificación de caracteres, como ASCII o Unicode
- Los **archivos binarios**:
 - Contienen datos en un formato que no es legible por humanos
 - El formato está compuesto por secuencias de bytes que pueden representar una amplia gama de tipos de datos (números, imágenes, audio, vídeo, ...)

Índice



- 1. Introducción**
- 2. Apertura y cierre de archivos**
- 3. Lectura de archivos de texto**
- 4. Manejo de errores**
- 5. Escritura de archivos de texto**

Apertura y cierre de archivos

Procedimiento de interacción con un archivo

- En Python (y otros lenguajes), el procedimiento habitual para **manejar archivos** desde un programa tiene **3 etapas**:
 - 1. Abrir** el archivo: se deben especificar el **nombre** del archivo (ruta) y el **modo** de operación (normalmente lectura o escritura)
 - 2. Realizar operaciones de lectura y/o escritura**, dependiendo del modo en el que se haya abierto el archivo
 - 3. Cerrar** el archivo, lo cual garantiza que las posibles operaciones pendientes se realizan y el archivo queda guardado de manera estable

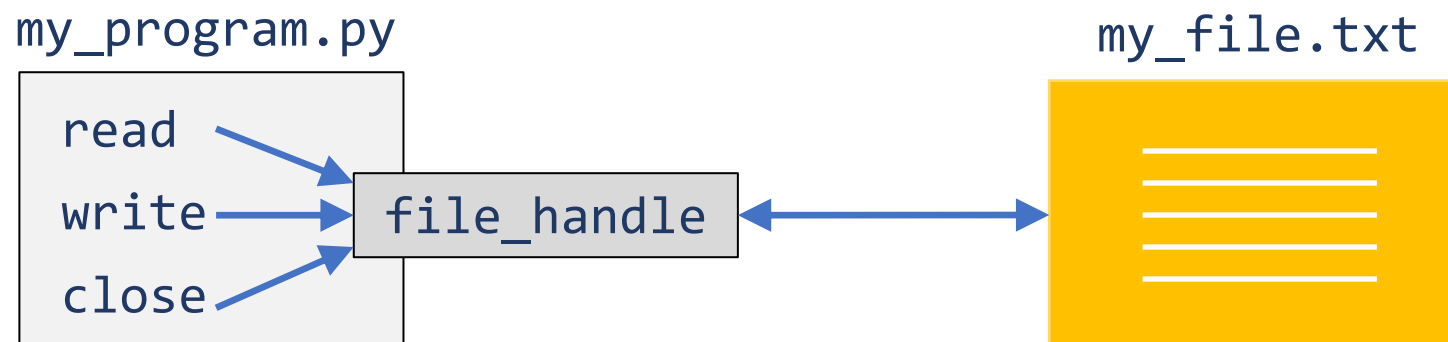
Apertura y cierre de archivos

Identificador (o *handle*) de un archivo

- En Python, la **interacción** entre el programa y cualquier archivo se realiza a través de una variable denominada **identificador** de archivo (***file handle***)
- La función predefinida **open () abre** un archivo y devuelve su identificador de archivo para su posterior manipulación:

```
file_handle = open('my_file.txt', 'mode')
```

- Una vez el archivo está abierto, cualquier **operación** (de lectura, escritura, etc.) sobre el archivo se realiza a través de su identificador:



Apertura y cierre de archivos

La función open ()

- Para **abrir** un archivo se usa la función **open ()**
- Esta función tiene **dos argumentos** (de tipo cadena):
 - La **ruta** del archivo, que puede ser absoluta o relativa
 - El **modo de apertura**, que puede ser uno de los siguientes:

Modo de apertura	Explicación
'r'	Abre el archivo para lectura (es el modo predeterminado si se omite el argumento)
'w'	Abre el archivo para escritura . Si el archivo ya existe, se sobrescribirá . Si no, se creará un nuevo archivo
'a'	Abre el archivo para anexar datos . Al escribir datos, estos se agregarán al final del archivo

Apertura y cierre de archivos

El método `close()`

- Cuando el programa ha completado todas las operaciones de lectura/escritura con el archivo, este debe cerrarse mediante el método **`close()`** sobre el identificador del archivo:

```
# Abrir el archivo (para leer, en este ejemplo)
file = open('my_file.txt', 'r')

# Realizar algunas operaciones de lectura aquí

# Cerrar el archivo
file.close()
```

Podemos utilizar cualquier nombre de variable como identificador del archivo



Apertura y cierre de archivos

La sentencia `with`

- Como alternativa para gestionar el acceso a un archivo evitando tener que cerrarlo de manera explícita (`close()`) podemos usar un concepto de Python denominado **administrador de contexto**
- Los administradores de contexto se definen mediante la sentencia **`with`**, y se encargan de ejecutar ciertas acciones de entrada y de salida de un determinado bloque de código
- En el caso de los archivos, al usar un administrador de contexto se realiza un **cierre automático** del archivo al terminar el bloque de código que lo manipula, sin tener que usar el método `close()`

Apertura y cierre de archivos

La sentencia with

- El uso de un **administrador de contexto** para manipular un **archivo** modifica ligeramente la función `open()`:

```
# Abrir un archivo mediante un gestor de contexto
with open('my_file.txt', 'r') as file:
    # Bloque de código (!!indentado!!) en el que
    # escribiremos las instrucciones de acceso al fichero
    # a través de su identificador 'file'

# Fuera del bloque del with, el archivo se cierra automáticamente
```

- Este método es mejor, porque garantiza el cierre del archivo, incluso en presencia de excepciones durante su manipulación

Apertura y cierre de archivos

La sentencia `with`

- Podemos incluso **abrir varios ficheros** en el mismo gestor de contexto, y todos ellos se cierran al terminar el bloque:

```
# Abrir dos archivos mediante un gestor de contexto
with open('data.txt', 'r') as f1, open('output.txt', 'w') as f2:
    # Aquí irían operaciones de lectura sobre f1
    # y operaciones de escritura sobre f2
    #

# f1 y f2 se cierran automáticamente
```

Índice



- 1. Introducción**
- 2. Apertura y cierre de archivos**
- 3. Lectura de archivos de texto**
- 4. Manejo de errores**
- 5. Escritura de archivos de texto**

Lectura de archivos de texto

Iteración sobre las líneas del archivo

- Un archivo de texto puede considerarse, y podemos manipularlo, como una **secuencia de líneas** (almacenadas en disco)
- Podemos leer dichas líneas considerando que el **identificador** del archivo es una **estructura de datos** sobre la que podemos **iterar** para obtenerlas:

```
file_name = input('Enter file name: ')\n\nwith open(file_name, 'r') as f:\n    for line in f:\n        print(line.rstrip())
```

Usamos un bucle for para **iterar** sobre los **elementos** de f (**líneas**)

rstrip() elimina el '\\n' final de cada línea para evitar que se imprima por duplicado

Lectura de archivos de texto

Otras formas de leer un archivo

- Una vez abierto un archivo, existen **varios métodos** sobre su identificador de archivo para leer sus líneas:

Método	¿Qué lee?	¿Qué devuelve?	Ejemplo
<code>read()</code>	El contenido completo del archivo	Una cadena	<pre>file_content = f.read() print(file_content)</pre>
<code>readline()</code>	La siguiente línea del archivo	Una cadena	<pre>line = f.readline() print(line.rstrip())</pre>
<code>readlines()</code>	Todas las líneas hasta el final del archivo	Una lista de cadenas	<pre>file_lines = f.readlines() for line in file_lines: print(line.rstrip())</pre>

Lectura de archivos de texto

La lectura como proceso secuencial

- La **lectura** de un archivo es un **proceso secuencial**
- Al **abrir** el archivo, la posición de lectura se establece al **principio**
- Cada operación de lectura **avanza** la posición de lectura tantas posiciones como el número de caracteres leídos
- **No es posible retroceder**, a menos que volvamos a abrir el archivo, lo que restablecería la posición de lectura de nuevo al principio
- Cuando la posición de lectura llega al **final** del archivo, cualquier operación de lectura devuelve una **cadena vacía**

Lectura de archivos de texto

Ejemplo: lectura de un fichero CSV

```
def read_csv(file_name, delimiter=',', header=True):  
    data = []  
    with open(file_name, 'r') as f:  
        if header:          # Si el archivo tiene un encabezado,  
            f.readline()    # Descartar la primera línea  
  
        while True:  
            line = f.readline().rstrip() # Leer la siguiente línea  
            if line == '':              # si hemos llegado al final del archivo  
                break                  # acabamos el bucle (no hay más líneas)  
            else:  
                items = line.split(delimiter) # Dividimos la línea  
                data.append(items)            # Añadimos la lista a 'data'  
  
    return data
```

Índice



- 1. Introducción**
- 2. Apertura y cierre de archivos**
- 3. Lectura de archivos de texto**
- 4. Manejo de errores**
- 5. Escritura de archivos de texto**

Manejo de errores

Errores en la apertura de un fichero

- Cuando un archivo no se puede abrir (el archivo no existe o la ruta es incorrecta), se genera una excepción de tipo **FileNotFoundError**
- Podemos manejar esta excepción usando un bloque **try-except**
- Por ejemplo:

```
file_name = input('Nombre del fichero: ')
try:
    with open(file_name, 'r') as file:
        # Leer el fichero
except FileNotFoundError:
    print(f'Error: {file_name} no encontrado')
```

Índice



- 1. Introducción**
- 2. Apertura y cierre de archivos**
- 3. Lectura de archivos de texto**
- 4. Manejo de errores**
- 5. Escritura de archivos de texto**

Escritura de archivos de texto

Modos y métodos de escritura

- Existen varios **modos** de apertura para **escribir** en un fichero:

Modo de apertura	Explicación
'w'	Abre el archivo para escritura . Si el archivo ya existe, se sobrescribirá . Si no, se creará un nuevo archivo
'a'	Abre el archivo para anexar datos . Al escribir datos, estos se agregarán al final del archivo

- Para escribir, podemos usar el método `write()` sobre el identificador del archivo:

```
file_handle.write('text')
```

El texto debe ser una cadena
(puede ser con formato)

A diferencia de `print()`, `write()` **no añade un salto de línea** al final

Escritura de archivos de texto

Modos y métodos de escritura

- **Ejemplo:** escribir las tablas de multiplicar en un archivo

```
file_name = input('Enter a filename to store the multiplication table: ')
try:
    with open(file_name, 'w') as f:
        for n in range(1, 11):
            f.write(f'THE MULTIPLICATION TABLE OF {n}:\n')
            for i in range(1, 11):
                f.write(f'{n} x {i} = {n*i}\n')
except FileNotFoundError:
    print(f'Error opening {file_name}')
```


Escritura de archivos de texto

Lectura y escritura

- **Ejemplo:** programa que lee el contenido de un fichero y escribe en otro fichero las líneas que contienen un texto determinado:

```
file_name_in = input('Name of input file: ')
text = input('Text to search for: ')
file_name_out = input('Name of output file: ')
try:
    with open(file_name_in, 'r') as f_in, open(file_name_out, 'w') as f_out:
        for line in f_in:
            if text in line:
                f_out.write(line)
except FileNotFoundError:
    print(f'Error opening file')
```